

7. INTELIGENCIA ARTIFICIAL GENERATIVA

José J. Martínez P.

Septiembre 2026

7.1. Introducción

Está por demás hablar del impacto que produjo la liberación de ChatGPT el 30 de noviembre de 2022. Hablamos entonces de que los avances en IA en el área ahora conocida como IA Generativa, están marcando nuevos derroteros en la disciplina de la IA. Se dice que hay un antes y un después de 2018 cuando se desarrolló el concepto de “Transformer”, como una nueva estructura de ML para la generación de grandes modelos de lenguaje, MLLs.

Pero, lo más impactante, no solo fue el desarrollo de esta nueva tecnología, sino su utilización casi inmediata por decenas de millones de personas sin formación científica o tecnológica, distribuidas en muchas partes del mundo. Es la primera vez en la historia de la humanidad que se presenta este suceso, la utilización inmediata a escala planetaria de una nueva tecnología.

Desde los 80s se comenzó a trabajar en ML, especialmente enfocado hacia el reconocimiento de objetos o patrones, **con base en el concepto de características** (features), sin embargo, con Deep Learning de alguna manera se dispara su utilización, llevando al reconocimiento de los datos, como la fuente de conocimiento, pues en ellos está la información real, válida, de la interacción de humanos y máquinas con el ambiente, todo esto considerando las RNAs como su columna vertebral.

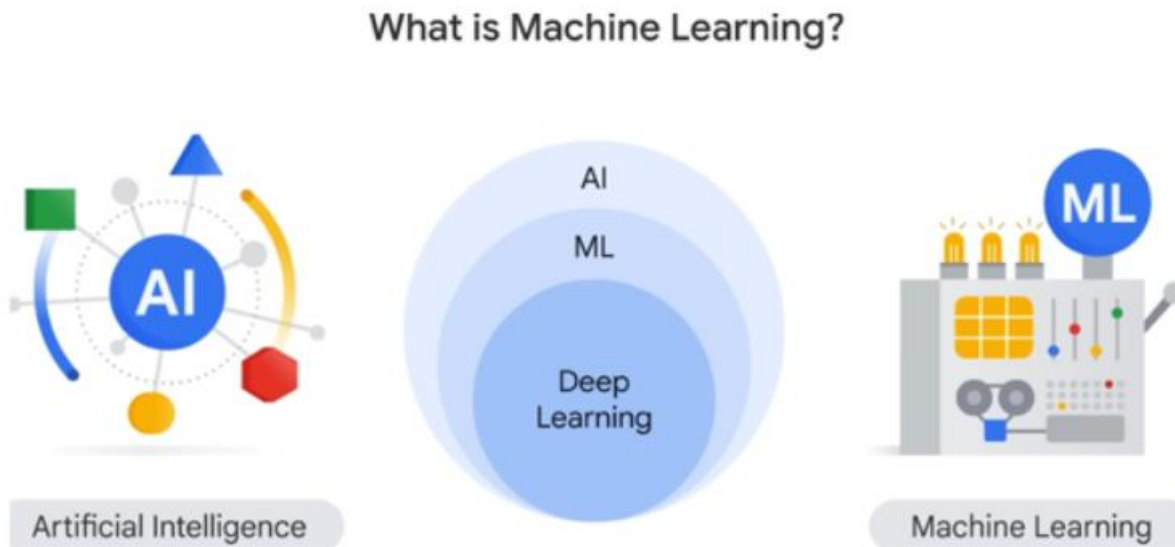


Figura 7.1. Relación entre IA, ML y DL.

7.2. IA Generativa

La IA Generativa es un tipo de tecnología de IA que puede producir diferentes tipos de contenido incluyendo texto, imágenes, audio, video y datos sintéticos; y lo hace a partir de

patrones aprendidos de datos existentes. La IA generativa es capaz de producir contenido altamente realista y complejo que imita la creatividad humana, a partir de indicaciones (prompts) hechas por personas que pueden no tener ningún conocimiento de tecnología, convirtiéndose en una herramienta valiosa en muchos campos del quehacer humano.

La historia de la IA Generativa, puede decirse que comenzó en la década de los 50s y 60s cuando los primeros investigadores comenzaron a explorar las posibilidades de la IA, enfocados en desarrollar sistemas basados en reglas que podían simular el pensamiento humano y la toma de decisiones. En los 90s emergieron las RNAs, inspiradas en el cerebro humano, capaces de aprender de los datos en una forma que no lo hacen los sistemas basados en reglas. Esto llevó a grandes avances.

Uno de los avances más significativo fue el desarrollo del Deep Learning, que permitió a la IA Generativa lograr nuevos niveles de realismo y creatividad en el último par de años, que se pueden atribuir a las siguientes tres razones:

Disponibilidad de datos. En el pasado los modelos de IA Generativa estaban limitados por la cantidad de datos de los cuales aprender. Ahora, con la disponibilidad de miles de millones de datos estos modelos aprenden de miles de millones de ejemplos, volviéndolos más precisos y realistas.

Avances en la capacidad de cómputo. La IA Generativa requiere mucho poder de cómputo para su entrenamiento, por lo que su disponibilidad ha permitido entrenar los modelos con data sets mucho más grandes con mejoras en la precisión y el realismo.

Nuevos algoritmos. Los investigadores han desarrollado nuevos algoritmos específicamente para IA Generativa, mucho más efectivos que otros algoritmos y procesos, lo que ha llevado a mejoras recientes en IA Generativa.

6.3 Comportamiento de las herramientas de IA Generativa

Para evaluar el comportamiento de las herramientas de IA Generativa se utilizan benchmarks. Un benchmark o prueba de referencia, es una medida de calidad que permite evaluar un desempeño contra otro. Así para evaluar el comportamiento de herramientas de ML existen una serie de data sets, benchmarks, de los cuales uno de los primeros fue MNIST ya mencionada, con números escritos a mano.

En un artículo de 2021, de Pott, Kiela et al., se presenta un gráfico de cómo la IA Generativa ha ido superando diferentes benchmarks, comparándola con el comportamiento humano para superar esos benchmarks. El gráfico parte de los 90s en el eje X; y en el eje Y, toma una medida normalizada del estimado del comportamiento humano que es la línea roja, en cero.

Así el reconocimiento de los números escritos a mano de MNIST que se lanzó en los 90s, tomó aproximadamente 20 años en sobrepasar el estimado del comportamiento humano. Switchboard tiene una historia similar, se lanzó en los 90s, su problema es pasar de voz a texto; tomó aproximadamente 20 años pasar la línea roja. ImageNet tiene una historia más reciente, se lanzó en 2009, tomó 10 años alcanzar el punto de saturación; y a partir de aquí

realmente se ha acelerado el paso de la línea roja, así SQUAD 1.1, para responder preguntas, se solucionó en 3 años. La respuesta fue SQUAD 2.0, que se solucionó en menos de 2 años.

Y luego el benchmark GLUE, que es un conjunto grande de tareas destinado a probar el estrés de los mejores modelos. Cuando se anunció a muchos de los científicos del campo les preocupó que fuera muy difícil para los modelos actuales. Pero GLUE se saturó en menos de 1 año. La respuesta fue SuperGlue destinado a ser mucho más difícil y también se saturó en menos de 1 año.

Benchmarks saturate faster than ever

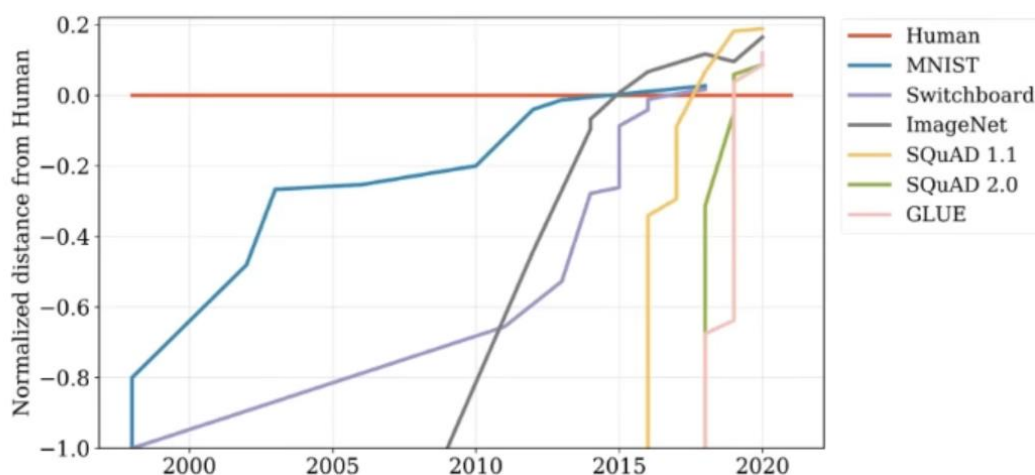


Figura 7.2. Cada vez es más rápido que las nuevas herramientas de IA Generativa, saturan los benchmarks.

Esta es una historia notable de progreso indudable, aun si se tienen dudas sobre las medidas del comportamiento humano, incluso aquí vemos un rápido incremento en la tasas de cambio. Y ahora 2021 fue hace años en la historia de la IA, específicamente en términos de NLU (Natural Language Understanding).

7.4. Modelos Fundamentales.

Los grandes modelos de lenguaje LLMs como ChatGPT, Gemini, DeepSeek, y otros, son realmente una parte de una clase diferente de modelos llamada modelos fundamentales. El término fue acuñado por el grupo de investigación de NLP en Stanford, cuando se dieron cuenta que este campo de la IA se estaba convirtiendo en un nuevo paradigma.

Las aplicaciones de IA estrechas se construyen entrenando cada una con data sets muy específicos del dominio, que producen diferentes modelos de ML, cada uno para una tarea específica, traductores, reconocimiento de números escritos a mano, filtros Spam.

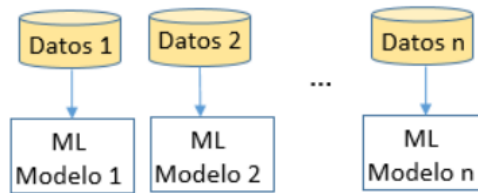


Figura 7.3. Se tienen aplicaciones de IA estrecha producidas con algún algoritmo de ML, un modelo específico, cada una con su propio data set, para propósitos específicos,.

Se predijo que estábamos yendo a un nuevo paradigma, donde tendríamos una capacidad fundamental o modelo fundamental, que podría incluir todos estos mismos casos de uso y aplicaciones. Así, las mismas aplicaciones que se proveían con estos modelos, podrían ahora construir los mismos modelos y un número de aplicaciones adicionales. El punto es que a este nuevo modelo se le puede transferir cualquier número de tareas. Lo que lleva a que este modelo tiene el superpoder de ser capaz de trabajar múltiples tareas y efectuar múltiples funciones.

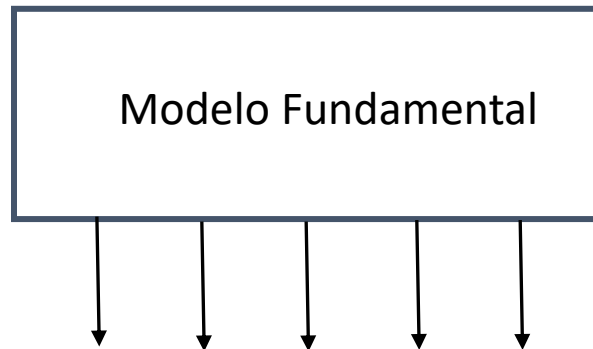


Figura 7.4. Modelo fundamental, incluye todos los data sets de todos los modelos estrechos, más datos no rotulados. Puede entonces, dependiendo de la afinación producir los modelos de IA estrecha, además de muchos otros.

Un modelo fundamental es un algoritmo de DL pre-entrenado con data sets extremadamente grandes obtenidos de internet. No son como los modelos de IA estrechos, entrenados para efectuar una sola tarea, pues se entrenan con una amplia variedad de datos y pueden transferir conocimiento de una tarea a la otra. Este tipo de RNA se entrena una vez y luego se afina para realizar diferentes tipos de tareas.

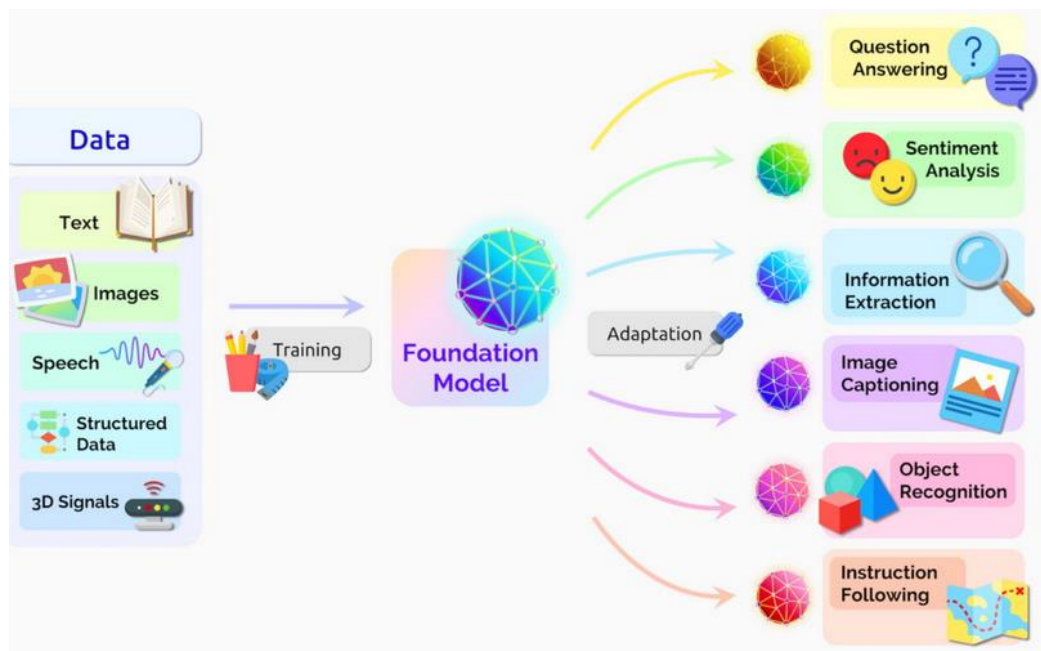


Figura 7.5. Construcción y utilidad de un modelo fundamental.

Estos modelos, figura 7.5, cuestan millones de dólares debido a que contienen cientos de miles de millones de hiperparámetros que se han entrenado con cientos de gigabytes de datos. Sin embargo, una vez terminado, cada modelo fundamental se puede modificar un número ilimitado de veces para automatizar un número ilimitado de tareas discretas.

Los modelos fundamentales actuales se usan para entrenar aplicaciones de IA que se basan en procesamiento de lenguaje natural. NLP, y la generación de lenguaje natural, NLG. Su uso más popular incluye los modelos Claude, GPT-4, DeepSeek y DALL-E 2.

7.5. Transformers

Los modelos fundamentales utilizan la arquitectura Transformer que es un tipo de arquitectura de RNA que ha ganado mucha popularidad en el campo de la IA Generativa. El desarrollo de los transformers estuvo dirigido a abordar el problema de la generación de una secuencia de texto, que incluye tareas como traducción, reconocimiento de voz, conversión texto a voz y más. Pero antes de entrar en detalle, veamos un par de conceptos claves para una mejor comprensión de la arquitectura Transformer. Token y embedding.

Token

Un token es una unidad de datos que viene de descomponer fragmentos más grandes de información, que puede representar palabras, caracteres o frases. Cuando se procesa texto, una frase se divide en tokens, donde cada palabra o signo de puntuación se considera un token separado. El proceso de tokenización es crucial en la preparación de los datos para el proceso posterior en modelos de IA. Los tokens no están restringidos solo a texto, pueden representar diferentes formatos de texto y juegan un papel fundamental en la habilidad de los LLMs en el aprendizaje y la comprensión.

Los tokens son fundamentales para que los LLMs interpreten la entrada y predigan la salida, manteniendo el contexto dentro de una ventana fija. El proceso típico incluye:

- **Tokenización.** El modelo divide el texto de entrada en tokens. Este proceso es una parte básica del procesamiento de lenguaje natural.
- **Tipos de tokens**
 - Tokens de texto. Se usan en los LLMs para generar respuestas similares a las de un humano, en diferentes aplicaciones.
 - Tokens de imágenes. Se aplican en modelos como Dall-e y Stable Diffusion donde se dividen las imágenes en estructuras similares, para generación de arte.
 - Tokens de audio. Se utilizan en modelos de voz de IA, donde las palabras habladas se convierten en representaciones tokenizadas para procesamiento y generación.

La mayoría de los tokens se manejan para mejorar las capacidades de estos modelos, permitiendo un procesamiento y generación más eficiente del lenguaje humano. Los modelos de IA procesan tokens para aprender las relaciones que hay entre ellos y desbloquear capacidades de predicción, generación y razonamiento. Entre más rápido se puedan procesar los tokens, los modelos pueden aprender y responder más rápido.

Importancia de los tokens

Costo y eficiencia. El costo de uso de los LLMs, en su mayoría, se basan en el uso de tokens, entre más tokens, más costo computacional. Comprender los límites de los costos es crucial, pues afectan directamente los gastos operativos.

Longitud del contexto

El número de tokens que puede procesar un LLM afecta su habilidad para mantener el contexto y la coherencia en la respuesta.

Ajuste fino y entrenamiento

Los desarrolladores optimizan los modelos ajustando la forma en que se van procesar los tokens, para mejorar su fluidez y relevancia. Son los ladrillos de construcción de los LLMs.

Ejemplo de tokenización.

Texto original:

Hola, estoy probando la tokenización como lo hace ChatGPT.

Tokens (IDs numéricos):

[69112, 11, 82384, 3650, 4988, 1208, 4037, 42600, 8112, 781, 35905, 13149, 38, 2898, 13]

Número de tokens: 15

```
69112 --> 'Hola'
11    --> ', '
82384 --> ' estoy'
3650  --> ' prob'
4988  --> ' ando'
1208  --> ' la'
4037  --> ' token'
```

```

42600 --> 'ización'
8112  --> ' como'
781   --> ' lo'
35905 --> ' hace'
13149 --> ' Chat'
38    --> 'G'
2898  --> 'PT'
13     --> '.'
```

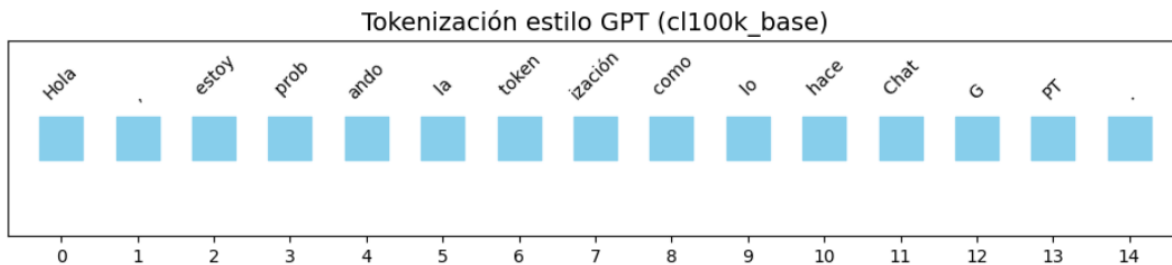


Figura 7.6. Tokenización estilo GPT.

Texto reconstruido:

Hola, estoy probando la tokenización como lo hace ChatGPT.

Funcionamiento de los tokens:

1. Diccionario fijo.

Cada modelo, por ejemplo, GPT-2, GPT-3, GPT-4 tiene su propio diccionario de tokens, que asigna a cada palabra, subpalabra o símbolo, un número entero único.

2. Universalidad dentro del modelo

Esto asegura que el mismo texto siempre se convierte en la misma secuencia de tokens, algo esencial para que el modelo pueda aprender y predecir.

Embeddings

Son representaciones vectoriales numéricas de datos complejos como texto, imágenes o audio, que usan los LLMs para comprender significados y relaciones. Al transformar estos datos brutos en números de punto flotante de más baja dimensionalidad, los embeddings capturan significado semántico y relaciones estructurales, permitiendo que los LLMs procesen, comparen y encuentren elementos similares, eficientemente.

El embedding de un token se obtiene al pasar su número por la capa de embedding del modelo, que consulta su matriz de embeddings, creando un vector con una determinada dimensión: 768, o 1024 u otra dependiendo del modelo, inicialmente único para cada token.

Veamos un ejemplo. Supongamos que tenemos los vectores Perro, Gato, Lobo, Robot, de 5 dimensiones:

- **Perro:** [0.8, 0.9, 0.1, 0.1, 0.2]
- **Gato:** [0.7, 0.8, 0.1, 0.2, 0.1]
- **Lobo:** [0.9, 0.5, 0.2, 0.1, 0.8]

- **Robot:** [0.1, 0.1, 0.9, 0.9, 0.1]

A cada dimensión le corresponde un concepto:

- **Dimensión 1: Grado de "Domesticación"** (Valores altos = Animal doméstico / Valores bajos = Artificial o salvaje)
- **Dimensión 2: Factor "Peludo / Orgánico"** (Valores altos = Ser vivo con pelo / Valores bajos = Metal, circuitos o sin pelo)
- **Dimensión 3: Grado de "Tecnología"** (Valores altos = Electrónico, mecánico / Valores bajos = Naturaleza pura)
- **Dimensión 4: Factor "Programable"** (Valores altos = Sigue un código o algoritmo / Valores bajos = Instinto o libre albedrío)
- **Dimensión 5: Factor "Salvaje / Salvajismo"** (Valores altos = Naturaleza hostil o indómita / Valores bajos = Entorno controlado)

Para calcular la distancia entre vectores en un embedding o espacio vectorial, se utilizan fórmulas que miden qué tan "ceranos" o "lejanos" están los puntos entre sí. Las dos métricas más comunes son la distancia euclidiana y la similitud de coseno. La más utilizada es la del coseno, que mide el ángulo entre dos vectores.

Si el resultado es 1, la similitud es total, si es 0, no hay ninguna relación, así se distribuyen los valores de similitud. Se divide el producto punto de los vectores entre la multiplicación de sus magnitudes.

$$\text{Similitud} = \frac{A \cdot B}{\|A\| \|B\|}$$

Relación	Distancia Euclidiana	Similitud de Coseno	Explicación
Perro y Gato	Muy Baja	Muy Cercana a 1	Ambos son mascotas domésticas y peludas.
Perro y Lobo	Media	Media-Alta	Comparten rasgos biológicos, pero el lobo es salvaje.
Perro y Robot	Muy Alta	Cercana a 0	No tienen casi nada en común en el contexto del lenguaje.

En los modelos de lenguaje reales (como GPT-4 o BERT), en lugar de 5 dimensiones, se utilizan entre 768 y 1536 dimensiones.

Para graficar vectores de 5 dimensiones en una pantalla de dos dimensiones (2D), en ciencia de datos se utilizan técnicas de reducción de dimensiones, la más común es la PCA (Análisis de Componentes Principales), la cual proyecta las 5 dimensiones en 2 ejes (X e Y) intentando perder la menor cantidad de información posible. Fue la que se utilizó aquí.

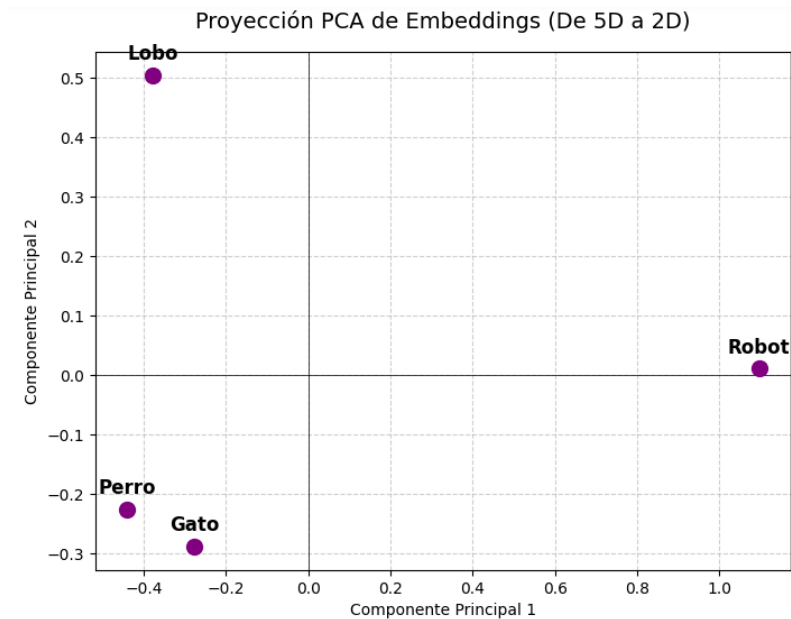


Figura 7.7. Cercanías entre los vectores Perro, Gato, Lobo y Robot.

Al combinar todos estos componentes, el MLL, puede “comprender” mejor el significado y generar texto más coherente y relevante. Los embeddings tienen cientos o miles de dimensiones y sus significados exactos no los decide un humano, sino que la máquina los aprende por su cuenta.

Al añadir más dimensiones, el modelo puede diferenciar con muchísima más precisión objetos que antes se parecían en tamaño o hábitat:

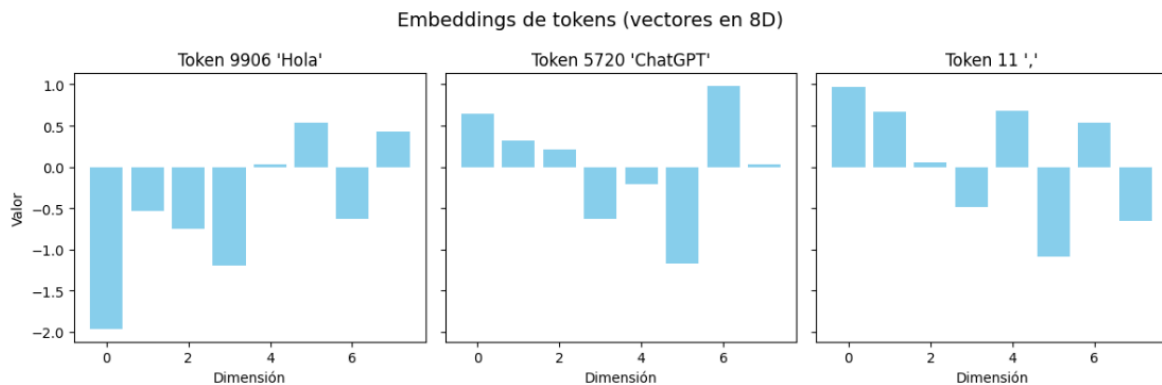


Figura 7.8. Vectores embedding de varias palabras

Sin embargo, los embeddings van evolucionando capa por capa para capturar el contexto y el significado de cada palabra. Los embeddings le dan significado semántico al texto.

En el artículo, “Attention Is All You Need” de T. Brown, et al, traducido, “El contexto es todo lo que usted necesita”, se plantea la arquitectura Transformer, figura 7.8, que solo se basa en los mecanismos de atención pues no incluyen mecanismos de recurrencia y convolución. Los mecanismos de atención se han convertido en la parte integral del modelado de secuencias

convincentes y transducción en varias tareas, pues permite el modelado de dependencias sin considerar sus distancias en las secuencias de entrada y salida.

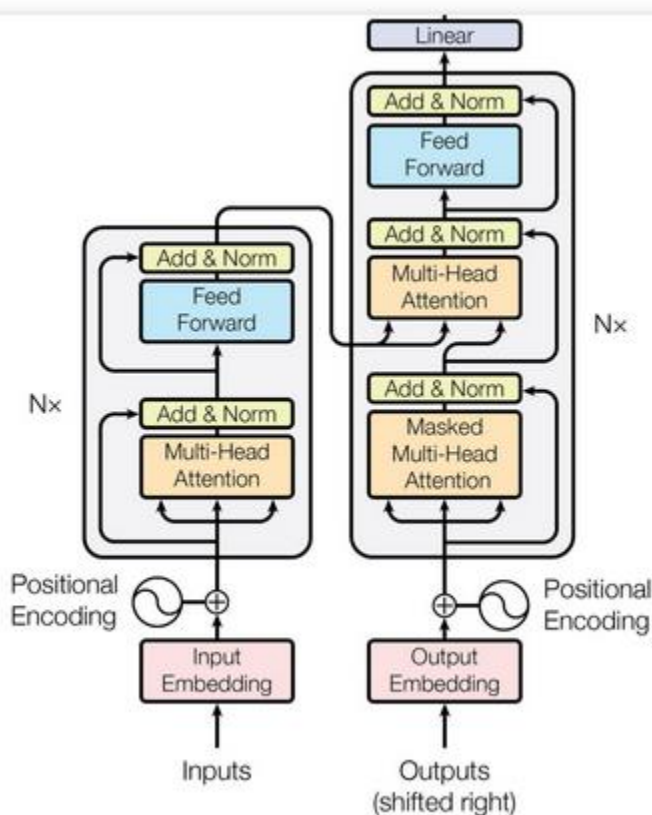


Figura 7.8. Arquitectura Transformer

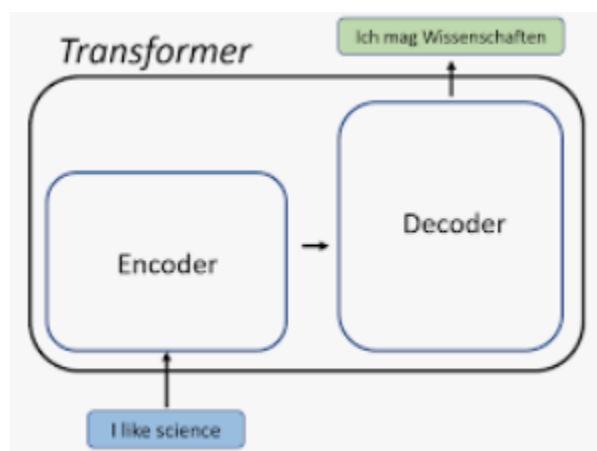


Figura 7.9. Arquitectura encoder-decoder

7.5.1 Encoder.

En un modelo transformer, el encoder, figura 7.9, permite al modelo comprender y codificar la secuencia de entrada, La arquitectura del encoder tiene varios componentes y procesos, con una pila de $N=6$ capas idénticas:

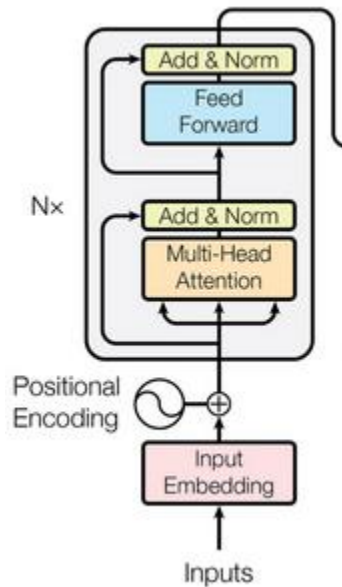


Figura 7.10. Arquitectura Encoder.

Embeddings. Los computadores solo comprenden datos numéricos, como vectores y matrices, por lo que es necesario convertir palabras, frases, imágenes, u otros, en vectores numéricos, generando un espacio vectorial, con contenido semántico. La idea, es que los elementos similares, en un dominio de aplicación específico, están muy cerca en ese espacio vectorial.

- Si hablamos de texto, el espacio embedding es un espacio donde cada palabra, a la cual se le ha asignado un valor, se agrupa con palabras similares.
- Si hablamos de imágenes, el embedding es un espacio de características visuales como forma, textura, color; lo que permite que el modelo comprenda conceptos visuales complejos.
- Si hablamos de audio y música, el embedding es un espacio de características como tono, ritmo, o timbre; lo que permite a los modelos generar música coherente.

Por ejemplo, los embeddings de texto, permiten medir la relación entre las cadenas de texto. Comúnmente se usan para:

- Búsqueda. Los resultados se clasifican por la pertenencia a un texto de consulta
- Agrupamiento. Se agrupan cadenas de texto por similitud.
- Recomendaciones. Se recomiendan ítems con cadenas de texto relacionadas.
- Detección de anomalías. Se identifican los valores atípicos porque tienen muy poca relación.
- Medidas de diversidad. Se analizan distribuciones de similitud.
- Clasificación. Las cadenas de texto se clasifican por los rótulos más similares.

Encoder posicional. Aborda el problema de la ambigüedad en las palabras del prompt, proveyendo el contexto con base en la posición de las palabras dentro de una frase. Entonces cada token se convierte en un vector embedding aumentado con un embedding posicional. Estos embeddings crean un vector de contexto. El encoder posicional permite al modelo comprender el significado de las palabras en diferentes contextos de las frases, ampliando su habilidad para capturar relaciones y dependencias en la secuencia de entrada.

Multi-head attention. El aspecto central de la arquitectura transformer es su mecanismo de “auto-atención”, que determina la importancia de cada palabra con respecto a las otras palabras dentro de una frase. Esto se logra generando un vector de atención para cada palabra que captura las relaciones contextuales entre las palabras de la frase. El uso de múltiples vectores de atención se conoce como bloque de atención multi-head.

Vectores Q, K y V

Son representaciones matemáticas de los tokens que permiten calcular la atención, el contexto. Así el modelo puede comprender qué palabras se relacionan entre sí y cuanta importancia se le debe dar a un token dentro de una frase. La idea es que en lugar de usar vectores x_i , directamente se representa por tres papeles separados que juega cada vector x_i .

Q, consulta: Como el elemento corriente que se está comparando con las entradas. Es lo que el token actual está “buscando” en el resto del texto para entender su propio contexto.

K, clave: Es el “rótulo” o descripción que presenta cada token del texto para ver qué tan relevante es con respecto a lo que buscan los demás tokens.

V, valor: Es el contenido real o significado del token.

Mecanismo de atención

El proceso ocurre en tres pasos matemáticos muy rápidos:

1. **El emparejamiento ($Q \times K$):** El modelo multiplica el vector **Query** de una palabra por los vectores **Key** de todas las demás palabras. Esto genera una puntuación de similitud.
2. **La normalización (Softmax):** Esas puntuaciones se transforman en porcentajes o “pesos de atención” que suman 100%. Por ejemplo, para la palabra “banco” en la frase “Fui al banco a sacar dinero”, la palabra “dinero” recibirá un peso de atención muy alto.
3. **La mezcla final (Pesos $\times$ V):** Los porcentajes obtenidos se multiplican por los vectores **Value** de cada palabra. El resultado es un nuevo vector enriquecido que sabe exactamente en qué contexto se encuentra la palabra.

Red feedforward

El cuarto paso incluye una red neuronal feedforward aplicada a cada vector de atención. La meta es transformar los vectores de atención en un formato que las capas subsecuentes del codificador o del decodificador puedan procesar. La red feedforward procesa los vectores de atención individualmente, uno a la vez. A diferencia de las Redes Neuronales Recurrentes, RNNs, los vectores son independientes uno de otro. Como resultado, se puede utilizar paralelización lo que mejora significativamente la velocidad de procesamiento y la eficiencia.

Con la habilidad para procesar los vectores de atención independientemente y en paralelo, podemos pasar todas las palabras al bloque encoder al mismo tiempo, produciendo un conjunto de vectores codificados para cada palabra, que se puede computar simultáneamente.

7.5.2 Decoder

La arquitectura del decoder, figura 4, lo mismo que la encoder, tiene varios componentes y procesos, con una pila de $N=6$ capas idénticas. En un modelo transformer, el decoder juega un papel

crucial en generar la secuencia de salida con base en la representación de la entrada codificada. Mientras que el encoder se enfoca en comprender la secuencia de entrada, el decoder se enfoca en generar la secuencia objetivo.

El ejemplo común es la traducción de una frase en inglés al francés. Se le provee una frase en inglés y la correspondiente traducción al francés para entrenar el modelo. La frase en inglés se procesa a través del bloque encoder, mientras que la frase en francés, la procesa el bloque decoder. Como el bloque encoder, el bloque decoder también incluye una capa embedding y un componente encoder posicional, que convierte las palabras de la frase de entrada en los vectores correspondientes. La arquitectura del decoder consiste de varios componentes y procesos que contribuyen a su funcionalidad como:

Masked multi-head attention

En este proceso, cada palabra de la frase interactúa dinámicamente con las palabras de su alrededor para descubrir las intrincadas relaciones entre ellas. El término “masked” se usa para evitar que el modelo se fije en las palabras que siguen durante este proceso. Continuando con el ejemplo, la secuencia de la entrada en francés se procesa a través del mecanismo de auto-atención. Este mecanismo es un componente fundamental que se usa tanto como vimos en el bloque encoder como en el bloque decoder, pues permite capturar las relaciones de dependencia entre diferentes palabras dentro de una frase o una secuencia.

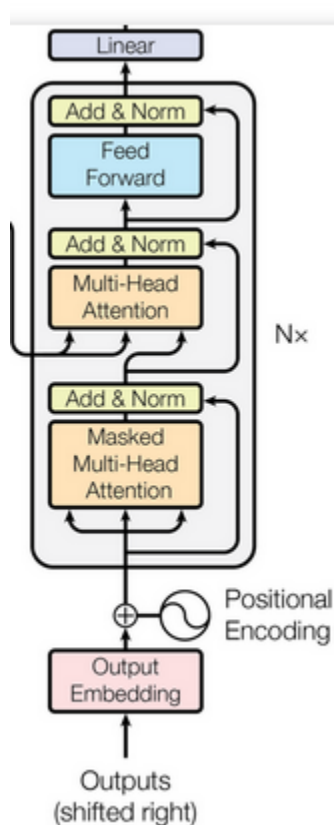


Figura 7.11. Arquitectura Decoder.

Para profundizar en el mecanismo de aprendizaje, durante el proceso de entrenamiento, el modelo primero predice la traducción al francés de cada palabra utilizando sus resultados anteriores. Estas

traducciones predichas luego se comparan con la traducción real en francés, que se proveyó como entrada al bloque decoder. Con base en la comparación, el modelo actualiza los valores de sus matrices, permitiéndole mejorar sus predicciones en cada iteración. Este proceso de aprendizaje iterativo continúa hasta que el modelo traduce *precisamente* las frases de entrada.

Durante el entrenamiento, se oculta o enmascara la siguiente palabra en francés. Esto asegura que el modelo debe aprender a traducir cada palabra al francés basado solamente en la palabra en inglés correspondiente y las palabras predichas al francés que lleva. Al enmascarar cada palabra en francés, evitamos que el modelo simplemente memorice los pares entrada-salida y lo anima a aprender patrones ocultos y relaciones entre los dos lenguajes. Cuando se procesa la frase en francés en el bloque decoder, solamente podemos usar las palabras previamente generadas en francés para predecir la palabra siguiente.

Par lograr esto, se enmascaran todas las futuras palabras en francés transformándolas en 0s. Esto se hace creando una matriz de máscara con la misma forma de la matriz de atención, con valores de 1 para la correspondiente palabra en francés que está disponible y 0 cuando esta enmascarada. Durante la operación de atención, se utiliza esta matriz de máscara para poner en 0 los pesos de atención, para las palabras enmascaradas en francés, asegurando que no contribuyen a la predicción de la palabra siguiente.

Multi-head attention block

El siguiente paso incluye pasar los vectores de atención obtenidos de la capa previa y los vectores codificados del bloque encoder a través de otro Multi-head attention block, donde se usan los resultados del bloque encoder. El bloque de atención encoder-decoder es donde sucede la principal proyección entre las palabras en inglés y en francés. Los bloques de atención que se generan en este bloque, capturan la relación contextual entre las palabras de la frase en inglés y aquellas en la correspondiente frase en francés.

Red feedforward

La unidad feedforward normalmente es una red neuronal simple de dos capas con una función de activación ReLu entre ellas. Se aplica a cada vector de atención independientemente, lo que permite la paralelización. Esta capa ayuda a transformar los vectores de atención en una forma más fácilmente digerible por la siguiente capa del modelo.

La salida de la capa lineal se pasa vía la capa softmax, produciendo una distribución de probabilidad sobre las posibles palabras de salida en francés. La función softmax, o función exponencial normalizada, es una generalización de la función logística, que se emplea para comprimir un vector K-dimensional Z , de valores reales arbitrarios, en un vector k-dimensional, $\sigma(Z)$, de valores reales en el rango $[0, 1]$. A cada palabra se le asigna una probabilidad con base en si la traducción reproduce fielmente la palabra inglesa introducida. Se escoge la palabra con más alta probabilidad como la palabra traducida para esa posición en la frase. Se repite este método para cada una de las palabras de la frase original en inglés para producir la frase traducida al francés.

7.6 Grandes modelos de lenguaje, LLM

Los LLMs son avances recientes en modelos DL para trabajar con lenguajes naturales humanos cuyo gran uso y utilidad se ha demostrado. Como humanos, percibimos el texto como una secuencia de

palabras. Así, una frase es una secuencia de palabras, los documentos son una secuencia de capítulos, secciones y frases. Para los computadores, el texto es solo una secuencia de caracteres con valor numérico. Para que un computador pueda “comprender” un texto, se puede construir un modelo con base en redes neuronales recurrentes. Este modelo procesa un carácter a la vez y provee una salida, una vez se haya procesado todo el texto. Este modelo trabaja bastante bien excepto algunas veces que “olvida” el comienzo de la secuencia cuando se llega al final del texto.

En 2017, Vaswani et al., publicaron el artículo, "Attention is All You Need", para establecer el modelo Transformer, que se basa en el mecanismo de atención. A diferencia de las redes neuronales recurrentes, el mecanismo de atención permite mantener toda la frase o incluso el párrafo a la vez, en lugar de una palabra cada vez. Esto permite al modelo Transformer comprender mejor el contexto de una palabra. Muchos modelos de NLP de última generación se basan en el modelo Transformer.

De acuerdo a C. Shannon, en su artículo “Prediction and Entropy of Printed English”, el lenguaje inglés tiene una entropía de 2.1 bits/letra, a pesar de que tiene 27 letras incluyendo el espacio. Si las letras se usaran aleatoriamente la entropía sería de 4.8 bits/letras ($\log_2 27$), lo que nos dice que, en los textos y los audios, no hay tanta aleatoriedad. Esto permite hacer más fácil el manejo computarizado que de un texto de lenguaje humano. Los modelos de ML y especialmente los modelos Transformer son expertos en utilizar esta baja entropía para hacer las predicciones de texto.

Pero, ¿cómo ve la gramática un modelo Transformer? Ya habíamos visto cómo es de complicado tratar de programar la serie de reglas de un lenguaje. En realidad, el modelo transformer no almacena estas reglas, las adquiere implícitamente a través de ejemplos. Es posible que el modelo aprenda mucho más que reglas gramaticales ampliando su conocimiento a ideas que viene en los ejemplos, obviamente el modelo transformer debe ser lo suficientemente grande.

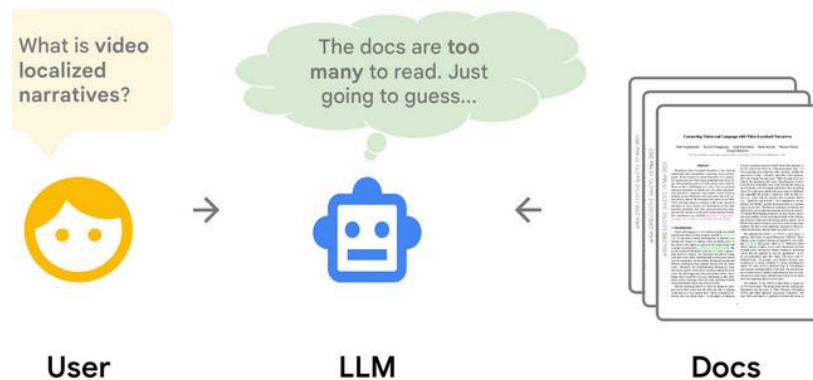
Se puede implementar un modelo Transformer desde cero utilizando una biblioteca de DL como TensorFlow o PyTorch. Esto permite tener una comprensión detallada de cómo funcionan estas arquitecturas.

7.7 RAG, Retrieval-Augmented Generation

RAG, Retrieval-Augmented Generation, recuperación por generación aumentada, es una técnica para mejorar la precisión y confiabilidad de los modelos generativos de IA con datos obtenidos de fuentes externas. Es una arquitectura de software que combina las capacidades de los LLMs, que tienen conocimiento general del mundo, con fuentes de información específicas para un negocio como: documentos, bases de datos SQL y aplicaciones internas del negocio. RAG mejora la precisión y la pertinencia de las respuestas de los LLMs.

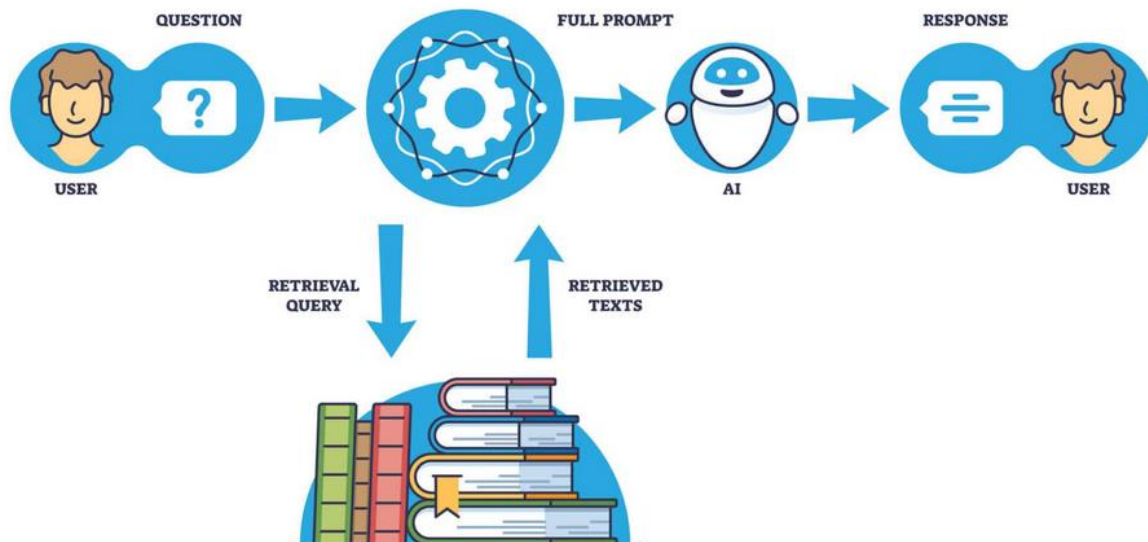
Cuando un MLL no tiene suficiente información, o no tiene conocimiento contextual de un tema, probablemente genere alucinaciones y dé respuestas imprecisas y falsas. El enfoque RAG puede ayudar a superar las limitaciones más significativas de los LLMs, como el conocimiento limitado a los datos de entrenamiento, la falta de contexto pertinente a los datos empresariales y datos no actualizados.

Challenge: LLM has limited knowledge, causing hallucinations

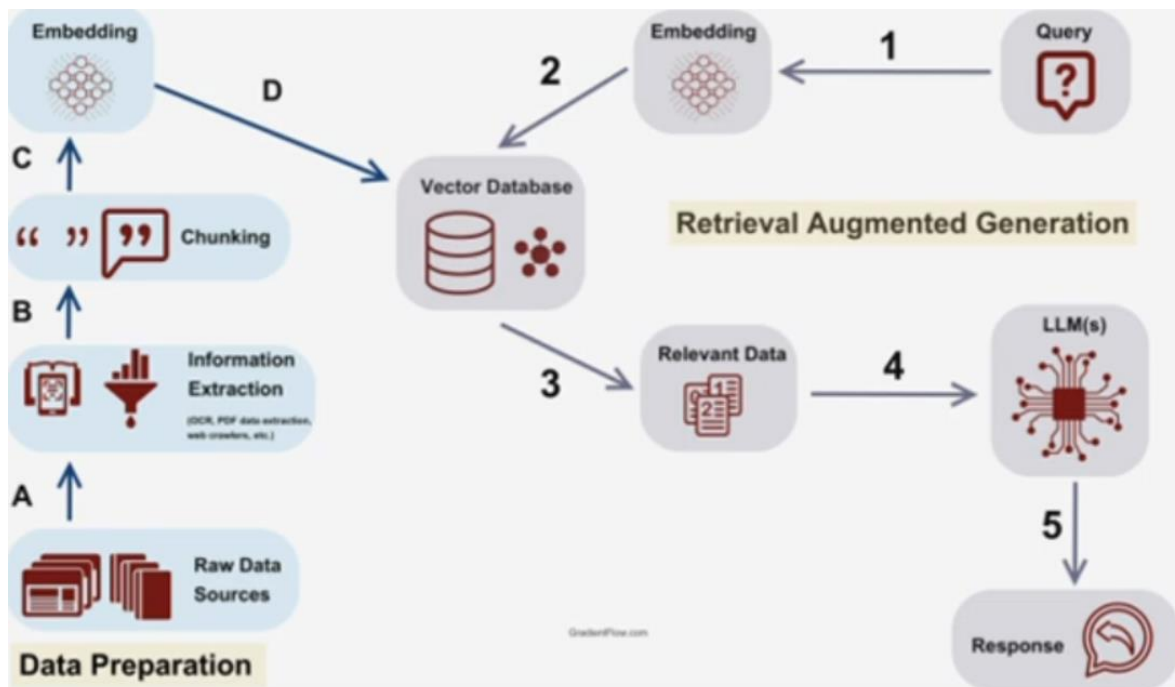


El enfoque RAG, en la medida en que se ha vuelto más popular, se ha hecho evidente que su eficacia depende completamente de la calidad de la búsqueda del sistema base de recuperación. Los LLMs son lo suficientemente inteligentes para comprender y responder preguntas, sin embargo, no se puede utilizar su poder si el sistema base no realiza búsquedas de alta calidad para escanear la gran cantidad de información propietaria.

RAG RETRIEVAL AUGMENTED GENERATION



RAG usa una tecnología de base de datos vectorial para almacenar información actualizada que se recupera usando búsquedas semánticas que se añade a la ventana de contexto del prompt, junto con otra información útil, para permitirle al LLM formular las mejores respuestas actualizadas.



Paso 1. Definición del problema, qué es lo que se quiere consultar sobre el conocimiento específico de una empresa, cual es el tipo de preguntas que va a responder el sistema y de ahí, ver los documentos fuente que se van a usar como base.

Entre las fuentes de datos están: documentos pdf, archivos de texto, páginas web, manuales técnicos, documentos científicos.

Paso 2. Extracción de información. Los documentos, en cualquier formato se deben cargar y convertirse en archivos de texto. Luego se debe hacer una limpieza y eliminar el ruido que puedan tener. Luego, debido a problemas de longitud de contexto, se dividen los documentos de texto en trozos llamados chunks.

Paso 3. Obtención de vectores embedding. Cada símbolo componente del chunk, se convierte en token. Ese token es un número y con ese número se recupera su vector de embedding. Se hace para todos los chunks. Estos vectores embedding se almacenan en una base de datos vectorial para su búsqueda semántica.

Paso 4. Consulta del usuario. Los símbolos del prompt del usuario primero se convierten en tokens y luego en se convierten en vectores embedding.

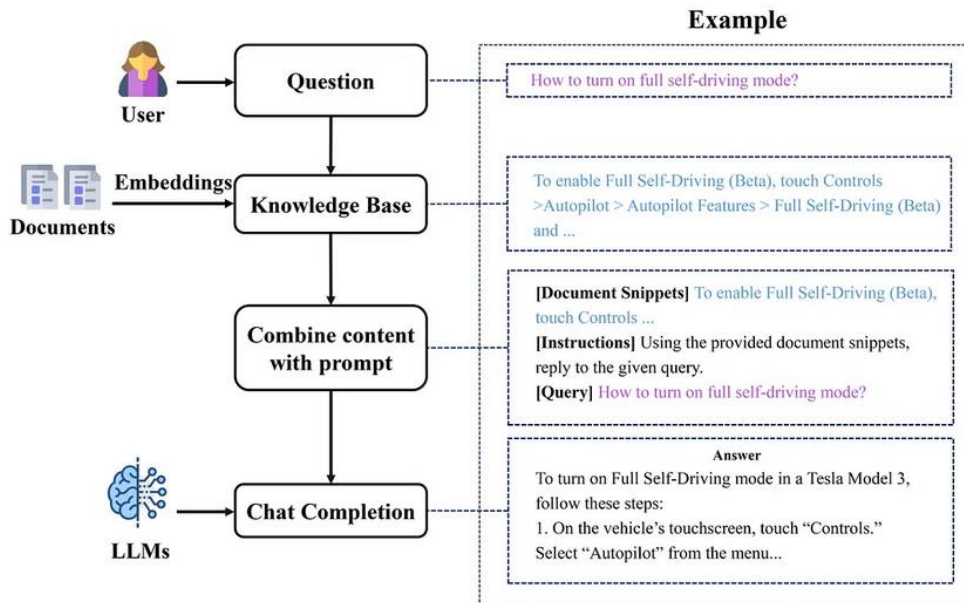
Paso 5. Recuperación de documentos relevantes. Se busca en la base de datos vectorial los chunks más cercanos. Los chunks más cercanos, se recuperan de la base de datos vectorial y se agregan al a los vectores embedding del prompt del modelo generativo.

Paso 6. Respuesta. El LLM, con base en esos vectores embeddings y su estructura transformer genera la respuesta.

7.7.1 Asistentes virtuales con RAG

En general las empresas e instituciones, pueden mejorar su imagen, utilizando asistentes virtuales para ampliar la interacción con los usuarios ofreciendo un servicio de alta calidad las 24 horas al día, siete días a la semana, con información completamente actualizada.

Por ejemplo, si la Universidad tuviera este tipo de servicio, su comunidad estaría enterada de la gran diversidad de ofertas y eventos científico-culturales que desarrolla permanentemente la Universidad, logrando con mayor efectividad sus objetivos misionales.



Flujo en un sistema RAG

7.8. Agentes

Los agentes de IA se definen como programas que pueden realizar tareas en nombre de los usuarios. Sin embargo, donde la situación se vuelve realmente interesante es con el concepto de sistemas multiagente, que implica que varios agentes de IA trabajen juntos para responder consultas o ejecutar flujos de trabajo, imitando la colaboración humana en equipo.

Los sistemas multiagente eficaces consisten en agentes especializados que pueden gestionar dependencias paralelas y ejecutar tareas de forma concurrente o secuencial. Existen plataformas y frameworks completos dedicados a la creación de estos sistemas multiagente, como LangChain y Crew AI.

En concreto, un agente de IA necesita tres cosas para ser útil:

1. Un modelo de IA para usar. Piensa en algo como chatGPT u otro LLM.
2. Memoria para que tenga contexto entre interacciones
3. Herramientas y Conocimientos para que sea capaz de investigar y brindar respuestas precisas

Por ejemplo, si le pregunta al agente cuál es la temperatura actual en Cali, para saber que ropa usar, el agente no podría responder. Necesitaría una conexión a un servicio meteorológico. De manera, que hay que proporcionarle al agente todas las herramientas necesarias para resolver su consulta. En una organización, una herramienta podría ser tu base de conocimiento, si está creando un agente de soporte, o quizá el sistema CRM si se trata de la gestión de cuentas.

Lo que nos lleva al MCP. Hablamos de herramientas, pero ¿cómo se conectan realmente estas herramientas? Sin duda, cada herramienta tiene su propia interfaz, y cada framework de agente tiene sus propias expectativas sobre cómo usarlas.

MCP es un protocolo abierto desarrollado por Anthropic para estandarizar cómo las aplicaciones proporcionan contexto (datos y herramientas) a los LLM. Se compara con un "puerto USB-C para aplicaciones de IA", que ofrece una forma estandarizada de conectar modelos de IA a diferentes fuentes de datos y herramientas.

El objetivo de MCP es abordar el panorama fragmentado actual, donde los desarrolladores crean integraciones personalizadas para que los LLM accedan a datos y herramientas de diferentes maneras. Entonces, en lugar de que cada uno aborde estas integraciones a su manera, existe una forma estandarizada en la que los desarrolladores pueden escribir un servidor MCP, que es básicamente una herramienta que los LLM pueden usar para diferentes tareas.

7.9. Hacia dónde va la IA generativa

Las actuales herramientas de IA Generativa han realizado avances notables pero es bueno anotar que se están en un estado primitivo de desarrollo. Aunque se han demostrado sus impresionantes capacidades, hay varias limitaciones y retos que se deben abordar y que actualmente están en investigación y es plausible que en los siguientes 5 a 10 años, evolucionará a límites sin precedentes, sobrepasando nuestras expectativas actuales.

Las tendencias en IA Generativa incluyen el crecimiento de Agentes de IA, capaces de actuar de forma independiente; la creciente sofisticación de la IA multimodal que procesa texto, imágenes y vídeo; y el auge de la hiper-personalización en servicios y contenido. Las empresas también están observando una adopción generalizada de la automatización de tareas y la aceleración de los flujos de trabajo creativos, a la vez que las consideraciones éticas y la necesidad de regulación cobran mayor relevancia.

- **Agentes de IA.** Un avance importante es la transición hacia Agentes de IA, que se refiere a sistemas de IA que pueden tomar la iniciativa y realizar tareas de forma autónoma sin intervención humana constante.
- **Hiper-personalización.** La IA Generativa facilita a las empresas la creación de servicios, productos y contenido de marketing altamente personalizados, adaptados a las necesidades individuales de los clientes en tiempo real.
- **Generación de contenido creativo.** La IA Generativa continúa acelerando los procesos creativos, desde la escritura y la generación de código hasta el diseño, el marketing y los esfuerzos artísticos.

- **Mejoras en IA conversacional.** Los avances en el procesamiento del lenguaje natural están haciendo que los chatbots y los asistentes virtuales sean más potentes y versátiles, lo que les permite gestionar consultas complejas de manera más eficiente.
- **IA física.** Integra modelos computacionales con el mundo real, permitiendo a máquinas y robots percibir, razonar y actuar en entornos tridimensionales basándose en leyes como la gravedad y la fricción. Tiene el potencial de revolucionar la forma en que concebimos, diseñamos y producimos productos físicos.

Veamos algunas formas que se pueden usar en conjunción con la impresión 3d:

- **Diseño generativo de Autodesk.** El software de diseño de Autodesk's Generative utiliza algoritmos de IA para generar diseños optimizados de productos con base en criterios específicos de comportamiento. Ya se ha aplicado en varias industrias como la automovilística y la aeroespacial, para crear componentes ligeros y estructuralmente eficientes.
- **Adidas Futurecraft Strung.** Adidas utilizó algoritmos de IA Generativa para crear la zapatilla Futurecraft Strung, que presenta una parte superior en forma de enrejado, construida a partir de datos generados por IA. Este enfoque permitió personalizar el ajuste y la sujeción, demostrando el potencial de la IA Generativa para personalizar productos físicos.
- **Nervous System's Kinematics Dress.** El estudio de diseño Nervous System ha utilizado la IA Generativa para crear el vestido Kinematics, una prenda impresa en 3D que se adapta y ajusta al cuerpo de quien la lleva. El vestido muestra cómo la IA Generativa puede revolucionar la industria de la moda con la producción de prendas complicadas y personalizables.
- **Optimización del diseño.** La IA generativa, puede asistir en la optimización de diseño de productos para impresión 3D. Introduciendo criterios de diseño y de comportamiento deseado, los algoritmos de IA Generativa pueden generar numerosas iteraciones de diseño. Estos diseños se pueden evaluar en función de factores como la reducción de peso, la integridad estructural, el uso de materiales y la eficiencia de la producción. Luego, se pueden ajustar los resultados y seleccionar para la impresión 3D.
- **Geometrías complejas y personalización.** La IA generativa permite crear geometrías complejas y personalizadas que se adaptan bien a la impresión 3D. Las técnicas de fabricación tradicionales tienen limitaciones a la hora de producir diseños complicados, pero la impresión 3D permite la realización de tales geometrías. Los algoritmos de IA Generativa pueden generar estructuras únicas y complejas que optimizan el uso de materiales, mejoran la funcionalidad o consiguen cualidades estéticas específicas. Este potencial de personalización permite fabricar productos personalizados adaptados a las necesidades y preferencias individuales.
- **Fabricación híbrida.** La IA generativa puede utilizarse para optimizar la integración de distintos materiales y procesos de fabricación con enfoques de fabricación híbridos. Por ejemplo, los algoritmos de IA Generativa pueden determinar dónde pueden combinarse técnicas de fabricación tradicionales como el mecanizado CNC o el moldeo por inyección con la impresión 3D para conseguir las propiedades y funcionalidades deseadas de un producto. Esta combinación de diferentes métodos de fabricación permite una mayor flexibilidad, eficiencia y rentabilidad en la producción.

7.10. Impacto social y ecosistema de modelos fundamentales.

(Tomado de On the Opportunities and Risks of Foundation Models, Rishi Bommasani et al, Center for Research on Foundation Models (CRFM) Stanford Institute for Human-Centered Artificial Intelligence (HAI) Stanford University). Pero lo que los hace críticos a modelos fundamentales es que se están integrando rápidamente en implementaciones de sistemas de IA en el mundo real, con consecuencias de gran alcance para las personas.

Pero, ¿cuál es la naturaleza de este impacto social? En este informe abordamos muchos aspectos de esta cuestión: la posible exacerbación de las desigualdades sociales (§5.1: equidad), el impacto económico debido al aumento de sus capacidades (§5.5: economía), el impacto ambiental debido a las mayores demandas de computación y energía (§5.3: medio ambiente), las posibles preocupaciones de ampliar la desinformación (§5.2: uso indebido), las ramificaciones legales debido a las poderosas capacidades generativas (§5.4: legalidad), las cuestiones éticas resultantes de la homogeneización y la economía política más amplia en la que se desarrollan y despliegan los modelos fundamentales (§5.6: ética).

Dada la naturaleza proteica de los modelos fundamentales y sus capacidades no proyectadas, ¿cómo podemos anticipar y abordar responsablemente las consideraciones éticas y sociales que plantean? Un tema recurrente es que es más fácil razonar sobre el impacto social de sistemas específicos implementados para usuarios específicos, que razonar sobre el impacto social de modelos fundamentales, que podrían adaptarse a cualquier número posterior de sistemas no previstos.

Si bien la producción de conocimiento puede desempeñar un papel vital en la configuración del futuro, el impacto social directo se produce a través de la implementación de estos modelos, que se rige por prácticas de propiedad sobre los datos, generalmente privados. A veces, la implementación se realiza a través de nuevos productos, pero a menudo se realiza mediante actualizaciones de productos existentes. Los modelos de investigación a menudo no se prueban exhaustivamente y pueden tener fallas desconocidas; deberían colocarse etiquetas de advertencia en los modelos de investigación que no sean aptos para implementarse. Por otro lado, los modelos fundamentales implementados que realmente afectan la vida de las personas deberían estar sujetos a pruebas y auditorías mucho más rigurosas.

Para comprender mejor la investigación y la implementación de los modelos fundamentales, debemos considerar el ecosistema completo en el que habitan estos modelos, desde la creación de datos hasta la implementación real. Es importante señalar que el modelo fundamental es sólo un componente de un sistema de IA. Simplificando, podemos pensar en el ecosistema de un modelo fundamental en términos de secuencia de etapas, extendiendo las etapas de capacitación y adaptación anteriores. Esta visión del ecosistema nos permite ver que diferentes preguntas sobre los modelos fundamentales en realidad deben responderse con respecto a las diferentes etapas.

(1) Creación de datos: la creación de datos es fundamentalmente un proceso centrado en el ser humano: todos los datos son creados por personas y la mayoría de los datos son, al menos implícitamente, sobre las personas. A veces, los datos son creados por personas para otras personas en forma de correos electrónicos, artículos, fotografías, etc., y a veces son una medición de personas (por ejemplo, datos genómicos) o una medición del entorno en el que viven las personas (por ejemplo, imágenes de satélite). Es importante tener en cuenta que todos los datos tienen un propietario y se crean con un propósito (donde ese propósito puede incluir o no el entrenamiento de un modelo fundamental).

(2) Conservación de datos: luego, los datos se seleccionan en datasets. No existe una distribución natural única de los datos; Incluso el rastreo de Internet más permisivo requiere cierta selección y filtrado posterior. Garantizar la relevancia y la calidad de los datos respetando al mismo tiempo las limitaciones legales y éticas es fundamental, pero desafiante. Si bien esto es reconocido en la industria, está subestimado en la investigación de la IA (§4.6: datos).

(3) Entrenamiento: El entrenamiento de los modelos fundamentales con en estos datasets seleccionados, es la pieza central celebrada en la investigación de la IA, aunque es solo una de muchas etapas.

(4) Adaptación: en el contexto de la investigación de ML, la adaptación consiste en crear un nuevo modelo basado en el modelo fundamental que realiza alguna tarea. Para la implementación, la adaptación consiste en crear un sistema, que requiere potencialmente muchos módulos diferentes, reglas personalizadas y combinación con otras señales. Por ejemplo, un modelo problemático capaz de generar contenido tóxico podría ser tolerable si se toman las precauciones adecuadas posteriormente. La lógica adicional, específica de la aplicación, es crucial para mitigar los daños.

(5) Implementación: el impacto social directo de un sistema de IA se produce cuando se difunde entre las personas. Aunque no quisiéramos implementar modelos fundamentales potencialmente dañinos, entrenados con datos cuestionables, aún podría ser valioso permitirlos en la investigación para avanzar en la comprensión científica, aunque aún hay que tener cuidado. En términos más generales, es una práctica estándar en implementaciones a gran escala realizar lanzamientos graduales, donde la implementación ocurre en una fracción cada vez mayor de usuarios; esto puede mitigar parcialmente cualquier daño potencial.

Si bien las grandes organizaciones pueden poseer todo el proceso, cada etapa podría ser realizada por una organización diferente, por ejemplo, una empresa que se especializa en la creación de modelos fundamentales personalizados para varios dominios que los desarrolladores de aplicaciones puedan utilizar. Piense en que el ecosistema, actúa como modelo. Si bien el impacto social depende de todo el ecosistema, sigue siendo importante poder razonar sobre las implicaciones sociales de un modelo fundamental, dado que el ámbito de muchos investigadores y profesionales se limita a la etapa de entrenamiento.

Esto es difícil porque los modelos fundamentales son objetos intermedios inacabados que pueden adaptarse a muchas aplicaciones posteriores, a veces por parte de una entidad completamente diferente para propósitos imprevistos. Lo que necesitamos son dos cosas: (i) métricas sustitutas para un conjunto representativo de posibles evaluaciones posteriores (§4.4: evaluación), y (ii) un compromiso para documentar estas métricas [Mitchell et al. 2019] similares a las hojas de datos para materiales como metales y plásticos, que pueden adaptarse a muchos casos de uso posteriores.

Caracterizar el posible impacto social posterior de los modelos fundamentales es un desafío y exige una comprensión profunda tanto del ecosistema tecnológico como de la sociedad. No se pueden evaluar plenamente los daños (§5.1: equidad) de un modelo fundamental sin reconocer cómo se implementará, y no se pueden simplemente definir métricas automáticas sin considerar el rico contexto social e histórico.

7.11. Ejercicios

En el contexto de IA un Skill es un paquete de folders de instrucciones, scripts y recursos, generalmente ancladas por un archivo Markown llamado Skill.md

1. Con base en los skills en <https://www.aihero.dev/> defina sus propias skills. En principio 4: 1. Un skill para documentar un ticket de trabajo, toda la descripción del trabajo por hacer. 2. Un skill de implementación, generación de código y pruebas. 3. Un skill de explicación, que genere un HTML que explique en detalle el trabajo realizado. 4. Un skill que lea y capture requerimientos de documentos provistos.

2. RAG industrial. Descargue varios manuales técnicos de máquinas que pueden estar en una planta y desarrollar un chatbot.2. Se sugiere usar Ollama.

3. Con base en los documentos entregados para el curso desarrollar un chatbot.

4. Desarrollar un agente de mantenimiento que:

- Busque un repuesto (Código)
- Consulte en el almacén (Código)
- Genere una orden de trabajo (falla, equipo)
- Actualice el historial de fallas

5. Suponga una aplicación usando un microcontrolador, por ejemplo un ESP32, podría usar un LLM? que restricciones tendría.

7.12 Bibliografía

J. Devlin et al, BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, fjacobdevlin,mingweichang,kentonl,kristoutg@google.com

T. Brown, Language Models are Few-Shot Learners, OpenAI

A. Vaswani, Attention Is All You Need, Google

S. Tedeschi, et al, What's the Meaning of Superhuman Performance in Today's NLU?

<https://www.youtube.com/watch?v=4Bdc55j80l8> Transformers agentic ai

[agentic ai - Google Search](#) Agentic AI

[Become a Real AI Hero](#) Skills